

Python e Software Engineering

Manuale didattico basato su un progetto reale

Assistente AI da riga di comando (CLI) con LLM locale/API e SQLite

Preparazione all'esame orale e pratico

Autore: rob

Ambiente: GNU/Linux · Python 3.10+

Indice

Introduzione: come usare questo manuale	3
PARTE I — Fondamenti di Python.....	4
1. Variabili, oggetti e riferimenti.....	5
2. Tipi di dato e mutabilit�	7
3. Operatori ed espressioni	9
4. Stringhe e f-string	11
5. Strutture dati: liste, tuple, set, dizionari.....	13
6. Condizioni e cicli	15
7. Comprensioni ed espressioni generatore.....	17
8. Funzioni, parametri e scope.....	19
9. Type hints (annotazioni di tipo)	21
10. Eccezioni e gestione degli errori	23
11. File e context manager	25
12. JSON.....	27
13. Moduli, package e import	29
14. Iteratori e generatori	31
PARTE II — OOP e progettazione	33
15. Classi e oggetti.....	34
16. Incapsulamento, ereditariet�, polimorfismo, composizione.....	36
17. Eccezioni personalizzate	38
18. Principi SOLID e design pattern	40
PARTE III — Analisi del progetto.....	42
19. Architettura e ciclo di vita	43
20. main.py.....	45
21. llm_client.py	47
22. tools/ (i cinque strumenti)	49
23. memory.py e il database.....	51
24. auth.py e la sessione	53
PARTE IV — Ingegneria del software	55
25. Organizzazione, dipendenze, virtual environment.....	56
26. Buone e cattive pratiche nel progetto.....	58
27. Testing e logging	59
28. Domande d'esame trasversali	60
Glossario	61

Introduzione: come usare questo manuale

Questo manuale ha un obiettivo doppio: farti **padroneggiare Python a livello teorico** e farti **capire a fondo il progetto** che porterai all'esame, in modo che tu possa rispondere a qualunque domanda spiegando concetti, ragionamento e scelte progettuali.

Il progetto usato come filo conduttore e' un **assistente AI da riga di comando**: un programma Python che, tramite un modello linguistico (LLM), riassume documenti, traduce testi, descrive immagini e genera ed esegue codice, con utenti e cronologia salvati su database SQLite. Ogni concetto teorico verra' prima spiegato in generale e poi mostrato "al lavoro" dentro questo progetto.

Per ogni argomento seguo sempre lo stesso schema, cosi' ti abitui a ragionare in modo ordinato:

1. **Definizione teorica** — cos'e'.
2. **Perche' esiste** — quale problema risolve.
3. **Come funziona internamente** — cosa succede "sotto il cofano".
4. **Sintassi Python** — come si scrive.
5. **Esempi semplici** — indipendenti dal progetto.
6. **Applicazione nel progetto** — dove e perche' compare.
7. **Errori comuni** — le trappole tipiche.
8. **Domande d'esame** — cosa potrebbe chiederti il professore e come rispondere.
9. **Riepilogo** — i punti chiave.

Da ricordare per l'esame

Quando spieghi qualcosa al professore, parti sempre dal *perche'*: "questo serve a...". Mostrare di aver capito la motivazione di una scelta vale piu' che ripetere la sintassi a memoria.

Legenda dei box

Approfondimento (azzurro): dettagli tecnici che dimostrano padronanza. **Attenzione** (arancione): errori e insidie. **Domande d'esame** (verde): possibili domande e risposte modello. **Da ricordare** (giallo): punti che conviene fissare.

PARTE I

Fondamenti di Python

1. Variabili, oggetti e riferimenti

1.1 Definizione teorica

Una **variabile** in Python e' un **nome** che fa riferimento a un **oggetto** in memoria. A differenza di linguaggi come il C, la variabile non e' "una scatola che contiene un valore": e' piuttosto **un'etichetta** attaccata a un oggetto. Ogni dato in Python (un numero, una stringa, una lista, una funzione) e' un oggetto con tre caratteristiche: un **identita'** (indirizzo in memoria, ottenibile con `id()`), un **tipo** (`type()`) e un **valore**.

1.2 Perche' esiste questo concetto

Il modello "nome → oggetto" permette a Python di essere dinamico: uno stesso nome puo' riferirsi in momenti diversi a oggetti di tipo diverso, e piu' nomi possono riferirsi allo stesso oggetto. Questo rende il linguaggio flessibile ma introduce il tema dei *riferimenti condivisi* (vedi mutabilita', cap. 2).

1.3 Come funziona internamente

Quando scrivi `x = 5`, Python crea (o riusa) l'oggetto intero `5` e fa puntare il nome `x` a quell'oggetto. L'assegnazione **non copia** l'oggetto: copia il riferimento. Gli oggetti non piu' riferiti da alcun nome vengono liberati dal **garbage collector** (in CPython, principalmente tramite *reference counting*: ogni oggetto tiene il conto di quanti nomi lo puntano).

1.4 Sintassi Python

```
x = 5                # x punta all'intero 5
nome = "Alice"      # assegnazione, nessuna dichiarazione di tipo
a = b = 0           # due nomi, stesso oggetto
x, y = 1, 2         # assegnazione multipla (tuple unpacking)
```

1.5 Esempi semplici

```
a = [1, 2, 3]
b = a                # b e a puntano ALLA STESSA lista
b.append(4)
print(a)             # [1, 2, 3, 4] -> modificata anche "a"
print(a is b)        # True: stessa identita'
```

1.6 Applicazione nel progetto

In `llm_client.py` i nomi a livello di modulo come `BACKEND`, `TEXT_MODEL`, `API_KEY` sono variabili che puntano a stringhe lette dall'ambiente. In `memory.py`, `conn = get_connection()` fa puntare il nome `conn` all'oggetto connessione al database, che poi viene usato e infine chiuso.

1.7 Errori comuni

Attenzione

Confondere **assegnazione** e **copia**. `b = a` su una lista non crea una nuova lista: modificando `b` modifichi anche `a`. Per copiare davvero servono `a.copy()` o `list(a)`.

1.8 Domande d'esame

Possibili domande

D: "In Python le variabili contengono valori?" — **R:** No: sono nomi che *referiscono* oggetti. L'oggetto vive nell'heap, il nome e' solo un'etichetta.

D: "Differenza tra `==` e `is`?" — **R:** `==` confronta i *valori*, `is` confronta l'*identita'* (stesso oggetto in memoria).

1.9 Riepilogo

Variabile = nome legato a un oggetto. L'assegnazione copia il riferimento, non l'oggetto. Tre proprieta' di ogni oggetto: identita', tipo, valore.

2. Tipi di dato e mutabilit 

2.1 Definizione teorica

Il **tipo** definisce quali valori un oggetto puo' assumere e quali operazioni supporta. Un oggetto e' **mutabile** se il suo contenuto puo' cambiare dopo la creazione (liste, dizionari, set), **immutabile** se no (int, float, bool, str, tuple, frozenset).

Categoria	Tipi	Mutabile?
Numerici	int, float, complex, bool	No
Testo	str	No
Sequenze	list / tuple	Si' / No
Insiemi	set / frozenset	Si' / No
Mappe	dict	Si'
Nulla	NoneType (None)	—

2.2 Perche' esiste questo concetto

La distinzione mutabile/immutabile e' cruciale per prevedere il comportamento del codice: gli immutabili sono sicuri da condividere (nessuno puo' modificarli "alle spalle" di un altro nome), i mutabili sono efficienti quando devi cambiare dati in loco ma richiedono attenzione ai riferimenti condivisi.

2.3 Come funziona internamente

Modificare un immutabile crea in realta' un **nuovo** oggetto: `s = s + "!"` costruisce una nuova stringa. Con un mutabile, `lista.append(x)` cambia lo stesso oggetto, senza crearne uno nuovo. Gli oggetti immutabili possono essere usati come chiavi di dizionario o elementi di set perche' sono **hashable** (il loro hash non cambia nel tempo).

2.4 Sintassi Python

```
type(3)          # <class 'int'>
isinstance(3, int) # True
x = None         # assenza di valore
```

2.5 Esempi semplici

```
s = "ciao"
s2 = s
s = s + "!"      # nuova stringa; s2 resta "ciao"
print(s, s2)     # ciao! ciao
```

```
t = (1, 2)
# t[0] = 9 -> TypeError: 'tuple' object does not support item assignment
```

2.6 Applicazione nel progetto

Il progetto usa quasi tutti questi tipi. La lista `messages` (mutabile) raccoglie i messaggi da inviare al modello. Le **tuple** immutabili sono usate come valore di ritorno "impacchettato": `run_file()` restituisce `(stdout, stderr, returncode)`. Il **set** `SUPPORTED_EXTENSIONS = {".jpg", ".jpeg", ".png"}` in `describe_image.py` rappresenta un insieme di formati validi. Il **dizionario** `LANG_EXTENSIONS` mappa linguaggi a estensioni. `None` e' usato come default di parametri opzionali (es. `max_length: int | None = None`).

Approfondimento

Perche' `run_file` restituisce una **tupla** e non una lista? Perche' il risultato ha una struttura *fissa* (sempre tre elementi con significato posizionale) e non deve essere modificato: la tupla comunica proprio "questo e' un record immutabile a tre campi".

2.7 Errori comuni

Attenzione

Usare un oggetto mutabile come **valore di default** di un parametro (`def f(x=[])`): il default viene creato una sola volta e condiviso tra le chiamate, causando bug. Il progetto evita l'insidia usando `None` come default (es. `tool_used=None`, `input_text=None`).

2.8 Domande d'esame

Possibili domande

D: "Perche' una tupla puo' essere chiave di un dizionario e una lista no?" — **R:** Perche' la chiave deve essere *hashable*, cioe' immutabile: la lista, essendo mutabile, non lo e'.

D: "Cosa succede a memoria quando concateni due stringhe?" — **R:** Si crea un nuovo oggetto stringa; gli originali restano invariati.

2.9 Riepilogo

Immutabili (`int`, `str`, `tuple`...) non cambiano: "modificarli" crea nuovi oggetti. Mutabili (`list`, `dict`, `set`) cambiano in loco. Solo gli *hashable* (immutabili) sono chiavi di `dict`/elementi di `set`.

3. Operatori ed espressioni

3.1 Definizione teorica

Un **operatore** e' un simbolo che combina uno o piu' valori (operandi) producendo un risultato. Le categorie principali: **aritmetici** (`+`, `-`, `*`, `/`, `//`, `%`, `**`), **di confronto** (`==`, `!=`, `<`, `>`, `<=`, `>=`), **logici** (`and` or `not`), **di identita'** (`is`, `is not`), **di appartenenza** (`in`, `not in`) e **di assegnazione** (`=`, `+=`, `...`).

3.2 Perche' esiste questo concetto

Gli operatori sono il modo conciso e leggibile di esprimere calcoli e decisioni. In Python molti sono **sovraccaricabili**: `+` somma numeri ma concatena liste e stringhe, a seconda del tipo degli operandi (polimorfismo, cap. 16).

3.3 Come funziona internamente

Gli operatori logici `and` / `or` usano la **valutazione pigra (short-circuit)**: `a and b` valuta `b` solo se `a` e' vero; `a or b` valuta `b` solo se `a` e' falso. Inoltre restituiscono **uno degli operandi**, non necessariamente un `bool`: `x or y` restituisce `x` se e' "veritiero", altrimenti `y`.

3.4 Sintassi Python

```
7 // 2      # 3 (divisione intera)
7 % 2       # 1 (resto)
2 ** 10     # 1024 (potenza)
a and b     # short-circuit
x or default # "x se veritiero, altrimenti default"
```

3.5 Esempi semplici

```
nome = ""
etichetta = nome or "anonimo" # "anonimo" (stringa vuota e' falsy)
print(3 < 5 <= 5)             # True (confronti concatenati)
```

3.6 Applicazione nel progetto

Il costrutto `model or API_MODEL` in `chat()` e' un uso idiomatico di `or`: "usa il modello passato, ma se e' `None` usa quello di default". Il `+` tra liste unisce contesto e nuovo messaggio: `context + [{"role": "user", ...}]`. Il confronto `password == row["password"]` in `auth.py` verifica le credenziali. La condizione `if max_length and len(summary) > max_length` sfrutta lo short-circuit: se `max_length` e' `None` (`falsy`), non valuta nemmeno il confronto.

3.7 Errori comuni

Attenzione

Confondere `==` (uguaglianza di valore) con `=` (assegnazione) e con `is` (identita'). Usare `is` per confrontare valori (es. `x is 5`) e' sbagliato: `is` va usato solo con `None` (`if x is None`), come fa correttamente il progetto.

3.8 Domande d'esame

Possibili domande

D: "Cosa restituisce `' '` or `'x'`?" — **R:** `'x'`, perche' la stringa vuota e' falsy e `or` restituisce il secondo operando.

D: "Cos'e' lo short-circuit e perche' e' utile?" — **R:** La valutazione si ferma appena il risultato e' deciso; utile per evitare calcoli inutili o errori (es. controllare che qualcosa esista prima di usarlo).

3.9 Riepilogo

Operatori aritmetici, di confronto, logici, identita', appartenenza. `and` / `or` sono pigri e restituiscono un operando. `is` solo con `None`.

4. Stringhe e f-string

4.1 Definizione teorica

Una **stringa** (`str`) e' una sequenza **immutabile** di caratteri Unicode. Le **f-string** (formatted string literals, da Python 3.6) sono stringhe con prefisso `f` in cui, tra parentesi graffe, si inseriscono espressioni valutate a runtime.

4.2 Perché esiste questo concetto

Le f-string rendono la costruzione di testo leggibile ed efficiente, sostituendo concatenazioni fragili (`"Ciao " + nome`) e il vecchio `%` o `.format()`. Sono fondamentali quando si costruiscono **prompt** per un LLM o messaggi per l'utente.

4.3 Come funziona internamente

Il compilatore trasforma la f-string in una serie di concatenazioni/conversioni già a compile-time: le espressioni `{...}` vengono valutate e convertite con `str()`. Le sequenze di escape come `\n` (a capo) sono interpretate; con il prefisso `r` (raw) invece no.

4.4 Sintassi Python

```
nome = "Alice"; n = 3
f"Ciao {nome}, hai {n} messaggi"      # "Ciao Alice, hai 3 messaggi"
f"totale: {10*n}"                    # espressione dentro le graffe
"riga1\nriga2"                      # \n = a capo
text.strip() text.lower() text.split(",") ", ".join(lista)
```

4.5 Esempi semplici

```
lingua = "inglese"
prompt = f"Traduci in {lingua}:\n\nCiao mondo"
print(len("ciao"))      # 4
print(" x ".strip())    # "x"
```

4.6 Applicazione nel progetto

Le f-string sono ovunque nella costruzione dei prompt: `f"Riassumi il seguente testo: \n\n{text}"` in `summarize.py`, `f"Traduci il seguente testo in {target_lang}..."` in `translate.py`. Sono usate anche per i messaggi di errore (`f"File non trovato: {path}"`) e per costruire nomi di file (`f"{source_path.stem}.{target_lang}.txt"`). Il metodo `.strip()` ripulisce la risposta del modello: `summary = llm_client.chat(messages).strip()`.

Approfondimento

Nel prompt `f"...:\n\n{text}"` il doppio `\n\n` separa l'istruzione dal contenuto: e' un piccolo accorgimento di *prompt engineering* che aiuta il modello a distinguere "comando" e "dati".

4.7 Errori comuni

Attenzione

Dimenticare la `f`: `"Ciao {nome}"` stampa letteralmente `{nome}`. Inoltre, poiche' le stringhe sono immutabili, metodi come `.strip()` o `.replace()` **non modificano** la stringa originale ma ne restituiscono una nuova: va riassegnata.

4.8 Domande d'esame

Possibili domande

D: "Le stringhe sono mutabili?" — **R:** No. `s.upper()` restituisce una nuova stringa, l'originale resta.

D: "Vantaggio delle f-string?" — **R:** Leggibilita' ed efficienza: l'espressione e' inline e valutata direttamente.

4.9 Riepilogo

Stringhe = sequenze immutabili di caratteri. Le f-string interpolano espressioni con `{}`. I metodi di stringa restituiscono nuove stringhe.

5. Strutture dati: liste, tuple, set, dizionari

5.1 Definizione teorica

Lista: sequenza ordinata e mutabile. **Tupla:** sequenza ordinata e immutabile. **Set:** insieme non ordinato di elementi unici. **Dizionario:** mappa chiave→valore, con chiavi uniche e hashable.

5.2 Perché esistono

Ognuna risolve un problema diverso: la lista per collezioni che cambiano; la tupla per record fissi; il set per test di appartenenza veloci e per eliminare duplicati; il dizionario per associare informazioni e cercarle per chiave.

5.3 Come funzionano internamente

Liste e tuple sono array di riferimenti; l'accesso per indice è $O(1)$. Set e dizionari sono **tabelle hash**: l'appartenenza (`in`) e la ricerca per chiave sono mediamente $O(1)$, contro l' $O(n)$ di una scansione di lista. Per questo un set è la struttura giusta per "questo elemento è tra quelli validi?".

5.4 Sintassi Python

```
lista = [1, 2, 3];      lista.append(4)
tupla = (1, 2, 3)      # immutabile
insieme = {"a", "b"};  "a" in insieme      # True,  $O(1)$ 
d = {"chiave": "valore"}; d["chiave"]; d.get("x", "default")
```

5.5 Esempi semplici

```
estensioni = {".jpg", ".png"}
print(".png" in estensioni)      # True
mappa = {"python": "py"}
print(mappa.get("rust", "txt")) # "txt" (default se assente)
```

5.6 Applicazione nel progetto

Lista di dizionari: `messages = [{"role": "system", ...}, {"role": "user", ...}]` è la struttura standard con cui si dialoga con un LLM. **Tupla:** `return translated, output_path` e `(stdout, stderr, returncode)`. **Set:** `SUPPORTED_EXTENSIONS` per verificare in $O(1)$ se un'estensione è valida (`path.suffix.lower() not in SUPPORTED_EXTENSIONS`). **Dizionario:** `LANG_EXTENSIONS.get(lang, "txt")` sceglie l'estensione con un default; ogni messaggio è un dict; le righe del DB diventano dict con `dict(row)`.

5.7 Errori comuni

Attenzione

Accedere a una chiave inesistente con `d["x"]` solleva `KeyError`; usa `d.get("x", default)` quando la chiave puo' mancare (come fa il progetto con `LANG_EXTENSIONS.get`). Ricorda che i set non mantengono l'ordine e non ammettono duplicati.

5.8 Domande d'esame

Possibili domande

D: "Perche' usare un set invece di una lista per i formati supportati?" — **R:** Perche' il test di appartenenza in e' $O(1)$ su un set e $O(n)$ su una lista, ed evita duplicati.

D: "Differenza tra `d['x']` e `d.get('x')`?" — **R:** Il primo solleva `KeyError` se manca, il secondo restituisce `None` (o un default).

5.9 Riepilogo

Lista (ordinata, mutabile), tupla (ordinata, immutabile), set (unici, hash, $O(1)$ su `in`), dict (chiave→valore, $O(1)$ in lookup). Scegli la struttura in base all'uso.

6. Condizioni e cicli

6.1 Definizione teorica

Le **istruzioni condizionali** (`if/elif/else`) eseguono blocchi diversi in base al valore di verita' di un'espressione. I **cicli** ripetono un blocco: `for` itera su una sequenza/iterabile, `while` ripete finche' una condizione resta vera.

6.2 Perche' esistono

Sono i costrutti di **controllo di flusso**: senza di essi un programma sarebbe una sequenza rigida. Le condizioni realizzano le decisioni, i cicli l'iterazione su collezioni (righe di un DB, pagine di un PDF, messaggi di una cronologia).

6.3 Come funzionano internamente

Python valuta la condizione secondo la **veridicita'** (truthiness): `0`, `""`, `[]`, `{}`, `None` sono *falsy*; quasi tutto il resto e' *truthy*. Il `for` chiede all'oggetto un **iteratore** (cap. 14) e ne consuma gli elementi fino a `StopIteration`. L'indentazione (di solito 4 spazi) delimita i blocchi: non e' estetica, e' sintassi.

6.4 Sintassi Python

```
if x > 0:
    ...
elif x == 0:
    ...
else:
    ...

for elemento in sequenza:
    ...
while condizione:
    ...
```

6.5 Esempi semplici

```
for i in range(3):
    print(i)          # 0 1 2
for c in "ab":
    print(c)          # a b
```

6.6 Applicazione nel progetto

La condizione centrale del progetto e' `if BACKEND == "api": ... else: ... in chat()` e `describe_image()`: sceglie il backend. In `auth.login()` una catena `if/elif/else` gestisce i tre casi (utente nuovo, password giusta, password errata). Il `for` compare in `cmd_history` (`for entry in entries:`) per stampare la cronologia e, in forma compatta,

dentro le comprensioni (cap. 7). La lettura del PDF itera sulle pagine: `for page in reader.pages`.

6.7 Errori comuni

Attenzione

Indentazione incoerente (mischiare tab e spazi) provoca `IndentationError / TabError`. Un altro classico: modificare una lista mentre la si itera. Il progetto evita il problema costruendo *nuove* liste con le comprensioni.

6.8 Domande d'esame

Possibili domande

D: "Cosa significa che un valore e' *falsy*?" — **R:** Che in un contesto booleano vale `False`: es. `0`, stringa vuota, lista vuota, `None`.

D: "Differenza tra `for` e `while`?" — **R:** `for` itera su un iterabile noto; `while` ripete finche' una condizione resta vera, utile quando il numero di iterazioni non e' noto a priori.

6.9 Riepilogo

`if/elif/else` per decidere, `for/while` per ripetere. Conta la truthiness. L'indentazione e' sintassi.

7. Comprensioni ed espressioni generatore

7.1 Definizione teorica

Una **comprensione** (list/dict/set comprehension) e' una sintassi concisa per costruire una collezione trasformando/filtrando gli elementi di un iterabile in un'unica espressione. L'**espressione generatore** ha sintassi simile ma produce gli elementi **pigramente**, uno alla volta, senza costruire la collezione in memoria.

7.2 Perche' esistono

Rendono il codice piu' breve e leggibile di un ciclo `for` con `append`, ed esprimono chiaramente l'intento "crea una lista trasformando X". Le espressioni generatore risparmiano memoria quando i dati sono tanti o servono una sola volta.

7.3 Come funzionano internamente

Una list comprehension e' equivalente a un ciclo che fa `append`, ma ottimizzato dall'interprete. Un'espressione generatore restituisce un **generatore** (cap. 14): non calcola nulla finche' non lo si itera, e produce un valore per volta.

7.4 Sintassi Python

```
[expr for x in iterabile if condizione]    # list comprehension
{k: v for ...}                             # dict comprehension
{expr for ...}                             # set comprehension
(expr for x in iterabile)                  # generator expression
```

7.5 Esempi semplici

```
quadrati = [n*n for n in range(5)]          # [0,1,4,9,16]
pari = [n for n in range(10) if n % 2 == 0]
somma = sum(n for n in range(1000))         # generatore: nessuna lista creata
```

7.6 Applicazione nel progetto

`memory.get_recent()` converte le righe del DB in dizionari con una list comprehension: `[dict(row) for row in reversed(rows)]`. In `get_connection()`, la migrazione ispeziona le colonne con `[row["name"] for row in conn.execute("PRAGMA table_info(users))"]`. In `summarize.extract_text()` l'estrazione del testo PDF usa un'**espressione generatore** dentro `join`: `"\n".join(page.extract_text() or "" for page in reader.pages)` — le pagine vengono elaborate una alla volta, senza costruire prima una lista completa.

Approfondimento

Perche' `join` con un generatore e non con una lista? Perche' `join` consuma comunque tutti gli elementi, ma il generatore evita di allocare una lista intermedia: piccola ottimizzazione di memoria che dimostra padronanza.

7.7 Errori comuni

Attenzione

Comprensioni troppo lunghe o annidate diventano illeggibili: in quei casi un ciclo esplicito e' preferibile. Ricorda che un generatore si consuma **una sola volta**: dopo averlo iterato e' esaurito.

7.8 Domande d'esame

Possibili domande

D: "Differenza tra `[x for x in ...]` e `(x for x in ...)`?" — **R:** La prima crea subito una lista in memoria; la seconda e' un generatore pigro che produce valori su richiesta.

D: "Quando preferire un generatore?" — **R:** Con grandi quantita' di dati o quando servono una sola volta, per risparmiare memoria.

7.9 Riepilogo

Comprensioni = costruire collezioni in una riga (con filtro opzionale). Espressioni generatore = versione pigra e leggera in memoria. Nel progetto: conversione righe DB e lettura pagine PDF.

8. Funzioni, parametri e scope

8.1 Definizione teorica

Una **funzione** e' un blocco di codice con un nome, che riceve **parametri**, esegue istruzioni e (opzionalmente) restituisce un valore con `return`. Lo **scope** e' la regione in cui un nome e' visibile; Python usa la regola **LEGB** (Local, Enclosing, Global, Built-in) per risolvere i nomi.

8.2 Perche' esistono

Le funzioni realizzano l'astrazione e il riuso: si scrive una logica una volta e la si richiama, con un nome che ne comunica l'intento. Riducono la duplicazione (principio DRY) e permettono di ragionare "a scatole" (input→output) senza pensare ogni volta ai dettagli interni.

8.3 Come funziona internamente

Alla chiamata, Python crea un **frame** con lo spazio dei nomi locali; i parametri diventano variabili locali legate agli argomenti passati. Gli argomenti sono passati **per riferimento di oggetto**: la funzione riceve lo stesso oggetto, quindi puo' modificarlo se e' mutabile, ma riassegnare il parametro non cambia la variabile del chiamante. I parametri possono essere posizionali, con valore di **default**, e variabili (`*args`, `**kwargs`).

8.4 Sintassi Python

```
def somma(a, b=0, *args, **kwargs):
    return a + b

def chat(messages, model=None, temperature=0.2, **options):
    ...
```

8.5 Esempi semplici

```
def saluta(nome, formale=False):
    return f"Buongiorno {nome}" if formale else f"Ciao {nome}"

saluta("Ada") # "Ciao Ada"
saluta("Ada", formale=True) # "Buongiorno Ada"
```

8.6 Applicazione nel progetto

Tutto il progetto e' organizzato in funzioni con responsabilita' unica: `extract_text`, `summarize_file`, `translate_file`, `generate`, `run_file`, `login`, ecc. La firma `def chat(messages, model=None, temperature=0.2, **options)` mostra parametri con default e

`**options` per inoltrare opzioni extra al backend locale. Il pattern `model or API_MODEL` gestisce il default "vero" quando il chiamante passa `None`. Le costanti a livello di modulo (`BACKEND`, `SYSTEM_PROMPT`) vivono nello scope **globale** del modulo e sono lette dalle funzioni.

Approfondimento — funzioni come oggetti

In Python le funzioni sono **oggetti di prima classe**: si possono assegnare a variabili e passare come argomenti. Il progetto lo sfrutta in `main.py`: ogni sotto-comando salva la propria funzione con `set_defaults(func=cmd_summarize)` e poi `main()` la invoca con `args.func(args)`. E' il pattern che elimina un lungo `if/elif` sui comandi.

8.7 Errori comuni

Attenzione

Default mutabili (cap. 2). Inoltre, dimenticare il `return` fa restituire `None` implicitamente. Modificare una variabile globale dentro una funzione richiede la parola chiave `global` (pratica in genere sconsigliata: meglio parametri e valori di ritorno, come fa il progetto).

8.8 Domande d'esame

Possibili domande

D: "Python passa gli argomenti per valore o per riferimento?" — **R:** Per *riferimento di oggetto* (pass-by-object-reference): la funzione riceve lo stesso oggetto; se e' mutabile puo' modificarlo, ma riassegnare il parametro non tocca il chiamante.

D: "Cos'e' la regola LEGB?" — **R:** L'ordine in cui Python cerca un nome: Local, Enclosing, Global, Built-in.

D: "A cosa serve `**options`?" — **R:** A raccogliere un numero variabile di argomenti con nome in un dizionario, per inoltrarli ad altre funzioni.

8.9 Riepilogo

Funzioni = astrazione e riuso. Parametri posizionali/default/variabili. Scope LEGB. Argomenti passati per riferimento di oggetto. Le funzioni sono oggetti: nel progetto sono usate come "valore" per il dispatch dei comandi.

9. Type hints (annotazioni di tipo)

9.1 Definizione teorica

I **type hints** (PEP 484) sono annotazioni che dichiarano il tipo atteso di parametri, valori di ritorno e variabili. Sono **facoltativi** e **non vincolanti a runtime**: Python non li verifica durante l'esecuzione, ma sono usati da strumenti esterni (mypy, IDE) e come documentazione.

9.2 Perché esistono

Migliorano leggibilità, autocompletamento e permettono di individuare errori di tipo *prima* di eseguire il programma (analisi statica). Comunicano il "contratto" di una funzione: cosa entra e cosa esce.

9.3 Come funzionano internamente

Le annotazioni sono salvate nell'attributo `__annotations__` ma **ignorate a runtime**. La sintassi `list[dict]`, `str | None` (unione di tipi, PEP 604) e' disponibile dalle versioni recenti di Python. Un *type checker* come mypy le legge e segnala incoerenze senza eseguire il codice.

9.4 Sintassi Python

```
def f(x: int, nome: str = "") -> bool: ...
valori: list[int] = []
opzionale: str | None = None
```

9.5 Esempi semplici

```
def media(neri: list[float]) -> float:
    return sum(neri) / len(neri)
```

9.6 Applicazione nel progetto

Il progetto e' **completamente annotato**, e questo e' esplicitamente premiato dalla griglia d'esame. Esempi: `def chat(messages: list[dict], model: str | None = None, temperature: float = 0.2, **options) -> str:`, `def run_file(path: str, timeout: int = DEFAULT_TIMEOUT, input_text: str | None = None) -> tuple[str, str, int]:`, `def current_user() -> dict | None:`. Le annotazioni dicono a colpo d'occhio cosa la funzione riceve e restituisce, e si integrano con le docstring (sezione Args/Returns).

9.7 Errori comuni

Attenzione

Credere che i type hint blocchino i tipi sbagliati a runtime: **non lo fanno**. `f("testo")` con `x: int` non solleva errori da solo; serve un type checker o un controllo esplicito. Vanno visti come documentazione verificabile, non come vincolo di esecuzione.

9.8 Domande d'esame

Possibili domande

D: "I type hint vengono controllati a runtime?" — **R:** No: sono ignorati durante l'esecuzione; servono a strumenti statici e come documentazione.

D: "Cosa significa `str | None`?" — **R:** Il valore puo' essere una stringa *oppure* `None` (tipo opzionale).

D: "Perche' annotare `-> tuple[str, str, int]`?" — **R:** Per dichiarare che la funzione restituisce una tupla di tre elementi con quei tipi, rendendo esplicito il contratto.

9.9 Riepilogo

I type hint documentano input/output senza vincolare il runtime. Il progetto li usa ovunque: e' una buona pratica esplicitamente valutata.

10. Eccezioni e gestione degli errori

10.1 Definizione teorica

Un'**eccezione** e' un oggetto che rappresenta un errore o una condizione anomala. Quando si verifica, il normale flusso si interrompe e Python cerca un gestore (`try/except`) risalendo lo stack. La **gestione degli errori** e' la strategia con cui il programma reagisce a queste situazioni.

10.2 Perché esiste

Separa il "codice normale" dalla "gestione dei guasti", rendendo il flusso principale leggibile. Permette di reagire in modo controllato (messaggio all'utente, ripristino, ritentativo) invece di far terminare il programma con un traceback incomprensibile.

10.3 Come funziona internamente

Le eccezioni sono oggetti che ereditano da `BaseException` (quasi sempre da `Exception`). `raise` le solleva; `try` delimita il codice sorvegliato; `except TipoErrore` cattura; `else` gira se non c'è stata eccezione; `finally` gira **sempre** (anche in caso di errore o `return`), ed è il posto giusto per rilasciare risorse. La clausola `raise ... from exc` costruisce una **catena** di eccezioni, conservando la causa originale.

10.4 Sintassi Python

```
try:
    rischioso()
except ValueError as e:
    print(e)
finally:
    pulisci()          # sempre eseguito

raise ValueError("messaggio")
raise LLMError("...") from exc  # concatenamento
```

10.5 Esempi semplici

```
def dividi(a, b):
    if b == 0:
        raise ValueError("divisione per zero")
    return a / b
```

10.6 Applicazione nel progetto

Il progetto ha una gestione degli errori curata. In `llm_client.py`, le chiamate al modello sono avvolte in `try/except` e gli errori tecnici vengono tradotti in una eccezione personalizzata `LLMError` con `raise LLMError(...) from exc`. In `memory.py`, `add_message` e

`get_recent` usano `try/finally` per garantire `conn.close()` anche in caso di errore. Le funzioni dei tool sollevano `FileNotFoundError` e `ValueError` per input non validi. Infine, `main()` cattura in un punto solo `LLMError` e `(FileNotFoundError, ValueError)`, stampando un messaggio pulito ed uscendo con `sys.exit(1)` invece del traceback.

```
try:
    args.func(args)
except llm_client.LLMError as exc:
    print(exc); sys.exit(1)
except (FileNotFoundError, ValueError) as exc:
    print(exc); sys.exit(1)
```

Approfondimento — perche' `from exc`

`raise LLMError(...) from exc` mantiene il collegamento all'errore originale (es. il problema di rete): nel traceback compare "The above exception was the direct cause...". Così si offre un messaggio pulito all'utente senza *perdere* l'informazione tecnica per il debug. E' una pratica avanzata che vale la pena citare all'esame.

10.7 Errori comuni

Attenzione

Catturare `except Exception` troppo genericamente nasconde bug reali. Meglio catturare i tipi specifici che ci si aspetta, come fa il progetto. Un altro errore: dimenticare di chiudere risorse — risolto con `finally` o con i context manager (cap. 11).

10.8 Domande d'esame

Possibili domande

D: "A cosa serve `finally`?" — **R:** A eseguire codice di pulizia in ogni caso, anche se c'è un'eccezione o un `return`; nel progetto chiude la connessione al DB.

D: "Perche' creare `LLMError` invece di sollevare l'errore originale?" — **R:** Per esporre al resto del programma un tipo di errore *di dominio* uniforme, gestibile in un punto solo, nascondendo i dettagli della libreria sottostante.

10.9 Riepilogo

`try/except/else/finally` per gestire gli errori; `raise` per sollevarli; `from` per concatenarli. Il progetto centralizza la gestione in `main()` e garantisce la chiusura del DB con `finally`.

11. File e context manager

11.1 Definizione teorica

Un **file** e' una risorsa esterna (su disco) che va aperta, usata e chiusa. Un **context manager** e' un oggetto che definisce cosa fare all'ingresso e all'uscita di un blocco `with`, garantendo il rilascio della risorsa anche in caso di errore.

11.2 Perche' esistono

Dimenticare di chiudere un file (o una connessione) causa perdite di risorse e dati non scritti. Il `with` automatizza la chiusura, rendendo il codice piu' sicuro e conciso rispetto a `try/finally` manuali.

11.3 Come funziona internamente

Il `with oggetto as x:` chiama `oggetto.__enter__()` all'ingresso (restituendo il valore legato a `x`) e `oggetto.__exit__()` all'uscita, sempre, anche se il blocco solleva un'eccezione. Un oggetto e' un context manager se implementa questi due metodi (il "protocollo" del context manager).

11.4 Sintassi Python

```
with open("file.txt", "r", encoding="utf-8") as f:
    contenuto = f.read()
# qui il file e' gia' chiuso automaticamente
```

11.5 Esempi semplici

```
with open("dati.txt", "w", encoding="utf-8") as f:
    f.write("ciao")
# nessun f.close() esplicito necessario
```

11.6 Applicazione nel progetto

In `llm_client._api_describe` l'immagine viene letta in modo binario con un context manager: `with open(image_path, "rb") as f: b64 = base64.b64encode(f.read()).decode()`. Il progetto usa anche l'astrazione di alto livello di `pathlib.Path`: `file_path.read_text(encoding="utf-8")` e `output_path.write_text(translated, encoding="utf-8")` aprono, leggono/scrivono e chiudono il file in una sola chiamata. La specifica esplicita di `encoding="utf-8"` garantisce la corretta gestione dei caratteri accentati su ogni sistema operativo.

Approfondimento

Anche l'oggetto connessione SQLite e' un context manager, ma il progetto sceglie `try/finally` con `conn.close()` per rendere esplicito il ciclo di vita della connessione: una scelta legittima e didatticamente chiara.

11.7 Errori comuni

Attenzione

Aprire file senza `with` e dimenticare `close()`. Non specificare `encoding`: su alcuni sistemi il default non e' UTF-8 e i caratteri accentati si rompono — il progetto lo specifica sempre.

11.8 Domande d'esame

Possibili domande

D: "Cosa garantisce il `with`?" — **R:** Che la risorsa venga rilasciata (file chiuso) all'uscita del blocco, anche in caso di eccezione.

D: "Quali metodi rendono un oggetto un context manager?" — **R:** `__enter__` e `__exit__`.

11.9 Riepilogo

Il `with` apre e chiude risorse in sicurezza. Nel progetto: lettura binaria dell'immagine, e lettura/scrittura testo via `pathlib` con encoding esplicito.

12. JSON

12.1 Definizione teorica

JSON (JavaScript Object Notation) e' un formato testuale per rappresentare dati strutturati (oggetti, array, stringhe, numeri, booleani, null). E' indipendente dal linguaggio ed e' lo standard di fatto per scambiare e salvare dati.

12.2 Perche' esiste

Serve a **serializzare** (trasformare in testo) strutture dati per salvarle su file o inviarle in rete, e a **deserializzare** (ricostruire gli oggetti dal testo). E' leggibile dall'uomo e mappa naturalmente su dizionari e liste Python.

12.3 Come funziona internamente

Il modulo standard `json` converte: `dict` $\leftrightarrow$ oggetto JSON, `list` $\leftrightarrow$ array, `str` $\leftrightarrow$ string, `int/float` $\leftrightarrow$ number, `True/False` $\leftrightarrow$ true/false, `None` $\leftrightarrow$ null. `json.dumps(obj)` produce una stringa; `json.loads(s)` ricostruisce l'oggetto.

12.4 Sintassi Python

```
import json
testo = json.dumps({"user_id": 1, "username": "alice"})
dati = json.loads(testo) # -> dict
```

12.5 Esempi semplici

```
import json
print(json.dumps([1, 2, {"k": None}])) # [1, 2, {"k": null}]
```

12.6 Applicazione nel progetto

La **sessione** in `auth.py` e' un file JSON. Al login: `SESSION_PATH.write_text(json.dumps({"user_id": user_id, "username": username}), ...)`. Per sapere chi e' loggato: `json.loads(SESSION_PATH.read_text(...))`. JSON e' perfetto qui perche' il dato e' un piccolo dizionario che deve sopravvivere tra un comando e l'altro (ogni comando e' un processo separato) ed essere leggibile.

12.7 Errori comuni

Attenzione

Passare a `json.loads` un testo non valido solleva `json.JSONDecodeError`. Il progetto lo gestisce: `current_user()` cattura `(json.JSONDecodeError, OSError)` e restituisce `None`, trattando una sessione corrotta come "assente" invece di crashare.

12.8 Domande d'esame

Possibili domande

D: "Differenza tra `dumps` e `loads`?" — **R:** `dumps` serializza (oggetto→stringa JSON), `loads` deserializza (stringa→oggetto).

D: "Perche' salvare la sessione in JSON e non in un database?" — **R:** E' un dato minimo e volatile; un file JSON e' semplice, leggibile e sufficiente, mentre il DB e' riservato a utenti e cronologia.

12.9 Riepilogo

JSON serializza strutture dati in testo. `dumps / loads` convertono da/verso dizionari e liste. Nel progetto memorizza la sessione utente.

13. Moduli, package e import

13.1 Definizione teorica

Un **modulo** e' un file `.py`: uno spazio dei nomi che raggruppa funzioni, classi e variabili correlate. Un **package** e' una cartella che contiene moduli (tradizionalmente con un file `__init__.py`). L'**import** e' il meccanismo con cui un modulo usa il codice di un altro.

13.2 Perche' esistono

Dividono un programma in unita' coese e riutilizzabili, evitando file monolitici. Favoriscono la **separazione delle responsabilita'**: ogni modulo ha un compito chiaro. Gli import esplicitano le dipendenze tra le parti.

13.3 Come funziona internamente

Al primo `import modulo`, Python esegue il file una volta, crea l'oggetto modulo e lo mette nella cache `sys.modules`; import successivi riusano la cache. La ricerca segue `sys.path`. Le forme sono: `import m` (usa `m.funzione`), `from m import f` (usa `f` direttamente), e l'**import "pigro"** (dentro una funzione) per caricare una libreria solo quando serve.

13.4 Sintassi Python

```
import memory
from tools import summarize, translate
from memory import get_connection, DB_DIR
```

13.5 Esempi semplici

```
# file util.py: def raddoppia(x): return 2*x
# altro file:
import util
print(util.raddoppia(4))    # 8
```

13.6 Applicazione nel progetto

Il progetto e' organizzato in moduli: `main.py`, `llm_client.py`, `memory.py`, `auth.py`, e il **package** `tools/` (con `__init__.py`) che raccoglie i cinque strumenti. `main.py` importa il necessario: `import auth, llm_client, memory` e `from tools import code_exec, codegen, describe_image, summarize, translate`. Un caso interessante e' l'**import pigro** in `llm_client.py`: `from openai import OpenAI` e `import ollama` sono dentro le funzioni, cosi' chi usa solo un backend non e' costretto ad avere installata la libreria dell'altro.

Approfondimento — import circolari

`auth.py` importa da `memory.py` (`get_connection`, `DB_DIR`): la dipendenza va in una sola direzione (`auth`→`memory`), evitando import circolari. Progettare le dipendenze come un grafo aciclico e' un principio di buona architettura.

13.7 Errori comuni

Attenzione

Import circolari (A importa B che importa A) causano errori difficili. Anche eseguire un modulo dalla cartella sbagliata rompe gli import relativi. Il progetto va lanciato dalla sua cartella con `python main.py`.

13.8 Domande d'esame

Possibili domande

D: "Differenza tra modulo e package?" — **R:** Il modulo e' un file `.py`; il package e' una cartella di moduli (con `__init__.py`).

D: "Perche' fare import dentro una funzione?" — **R:** Import "pigro": si carica la libreria solo se e quando serve; nel progetto evita di richiedere `openai` a chi usa solo Ollama e viceversa.

D: "Un modulo importato due volte viene eseguito due volte?" — **R:** No, viene messo in cache in `sys.modules` ed eseguito una sola volta.

13.9 Riepilogo

Modulo = file; package = cartella di moduli. Gli import esplicitano le dipendenze e sono messi in cache. Il progetto separa le responsabilita' in moduli e usa import pigri per i backend.

14. Iteratori e generatori

14.1 Definizione teorica

Un **iterabile** e' un oggetto su cui si puo' ciclare (lista, stringa, file, righe di un DB). Un **iteratore** e' l'oggetto che produce gli elementi uno alla volta tramite `next()`. Un **generatore** e' un modo semplice per creare iteratori: una funzione con `yield` o un'espressione generatore.

14.2 Perche' esistono

Permettono di elaborare sequenze potenzialmente grandi **senza caricarle tutte in memoria**, producendo un elemento per volta (valutazione pigra). Sono il meccanismo che sta dietro al ciclo `for`.

14.3 Come funziona internamente

Un `for x in iterabile` chiama `iter(iterabile)` per ottenere un iteratore, poi `next()` ripetutamente fino a `StopIteration`. Una funzione con `yield` non esegue subito il corpo: restituisce un generatore che, a ogni `next()`, riprende dall'ultimo `yield` mantenendo lo stato. Un generatore si consuma una sola volta.

14.4 Sintassi Python

```
def conta(n):
    for i in range(n):
        yield i          # produce un valore e "sospende"

somma = sum(x*x for x in range(10))  # espressione generatore
```

14.5 Esempi semplici

```
g = (c for c in "abc")
print(next(g))  # 'a'
print(next(g))  # 'b'
```

14.6 Applicazione nel progetto

Anche senza definire generatori con `yield`, il progetto li usa nella forma di **espressioni generatore** e di **iterazione pigra**. In `extract_text: "\n".join(page.extract_text() or "" for page in reader.pages)` — `reader.pages` e' iterabile e le pagine vengono prodotte man mano. Le query SQLite restituiscono cursori iterabili, consumati con `.fetchall()` o direttamente in comprensioni. Il ciclo `for entry in entries in cmd_history` itera sui risultati.

14.7 Errori comuni

Attenzione

Provare a riutilizzare un generatore già consumato: sarà vuoto. E confondere iterabile (ci si può ciclare più volte, come una lista) con iteratore (si esaurisce).

14.8 Domande d'esame

Possibili domande

D: "Cosa fa `yield`?" — **R:** Trasforma una funzione in generatore: produce un valore e ne sospende l'esecuzione, riprendendola alla chiamata successiva.

D: "Vantaggio di un generatore su una lista?" — **R:** Non tiene tutti gli elementi in memoria: li produce su richiesta, utile per grandi volumi o flussi.

14.9 Riepilogo

Iterabile → iteratore → `next()` fino a `StopIteration`. I generatori (con `yield` o espressioni) sono iteratori pigri. Il `for` ne è il consumatore naturale.

PARTE II

OOP e progettazione

Nota importante per l'esame

Il progetto e' scritto in stile **procedurale** (funzioni e moduli), con l'unica classe `LLMError` . La griglia d'esame valuta anche l'OOP: questa parte ti insegna la teoria e, soprattutto, ti prepara a rispondere se il professore chiede "come lo trasformeresti a oggetti?". Ogni capitolo mostra la teoria e come si applicherebbe a *questo* progetto.

15. Classi e oggetti

15.1 Definizione teorica

Una **classe** e' un modello (blueprint) che definisce attributi (dati) e metodi (comportamenti). Un **oggetto** (o istanza) e' un esemplare concreto creato da una classe. La programmazione orientata agli oggetti (OOP) organizza il codice attorno a oggetti che incapsulano stato e comportamento.

15.2 Perche' esiste

L'OOP modella entita' del dominio come "cose" con dati e azioni proprie, favorisce l'incapsulamento (nascondere i dettagli interni), il riuso (ereditarieta') e l'estensibilita'. E' particolarmente utile quando piu' dati e comportamenti sono strettamente legati.

15.3 Come funziona internamente

La classe e' essa stessa un oggetto. Alla creazione di un'istanza, Python chiama `__init__` (il costruttore) sul nuovo oggetto; `self` e' il riferimento all'istanza corrente, passato automaticamente ai metodi. Gli attributi vivono nel dizionario `__dict__` dell'istanza.

15.4 Sintassi Python

```
class Utente:
    def __init__(self, nome: str):
        self.nome = nome          # attributo di istanza
    def saluta(self) -> str:      # metodo
        return f"Ciao, sono {self.nome}"

u = Utente("Ada")
print(u.saluta())
```

15.5 Esempi semplici

```
class Contatore:
    def __init__(self):
        self.n = 0
    def incrementa(self):
        self.n += 1

c = Contatore(); c.incrementa(); print(c.n)    # 1
```

15.6 Applicazione nel progetto

L'unica classe definita e' `LLMError(RuntimeError)` in `llm_client.py`: una classe minimale ma didatticamente importante (vedi cap. 17). Il progetto **usa** molti oggetti creati da classi di libreria: la connessione `sqlite3.Connection`, il client `ollama.Client`, il client

`OpenAI`, gli oggetti `Path`. Anche `argparse.Namespace` (l'oggetto `args`) e' un'istanza con attributi (`args.username`, `args.func`).

Approfondimento — come si rifattorizzerebbe a oggetti

Una versione OOP potrebbe introdurre: una classe `LLMClient` che incapsula backend, modello e chiave con un metodo `chat()`; una classe `Database` che incapsula la connessione con metodi `add_message()` / `get_recent()`; una classe base astratta `Tool` con sottoclassi `SummarizeTool`, `TranslateTool`... (polimorfismo). Sapere *descrivere* questa evoluzione dimostra padronanza anche mantenendo il codice procedurale.

15.7 Errori comuni

Attenzione

Dimenticare `self` nei metodi; confondere **attributi di classe** (condivisi da tutte le istanze) con **attributi di istanza** (propri di ciascun oggetto). Creare classi "contenitore" senza comportamento quando basterebbe una funzione o una dataclass.

15.8 Domande d'esame

Possibili domande

D: "Differenza tra classe e oggetto?" — **R:** La classe e' il modello, l'oggetto e' l'istanza concreta creata da quel modello.

D: "Cos'e' `self`?" — **R:** Il riferimento all'istanza corrente, tramite cui i metodi accedono ai suoi attributi.

D: "Il tuo progetto usa l'OOP?" — **R:** In modo limitato: definisce l'eccezione `LLMError` e usa numerosi oggetti di libreria; la logica applicativa e' procedurale, ma sarebbe incapsulabile in classi come `LLMClient` e `Database`.

15.9 Riepilogo

Classe = modello, oggetto = istanza. `__init__` costruisce, `self` e' l'istanza. Il progetto definisce `LLMError` e usa molti oggetti di libreria.

16. Incapsulamento, ereditarieta', polimorfismo, composizione

16.1 Definizione teorica

I quattro pilastri/relazioni dell'OOP:

Concetto	Significato
Incapsulamento	Nascondere i dettagli interni ed esporre un'interfaccia; proteggere lo stato.
Ereditarieta'	Una classe (figlia) estende un'altra (madre), riusandone e specializzandone il comportamento (relazione "e'-un").
Polimorfismo	Oggetti di tipi diversi rispondono allo stesso messaggio (metodo) in modo proprio.
Composizione	Un oggetto ne contiene altri e delega loro dei compiti (relazione "ha-un").

16.2 Perche' esistono

Sono strumenti per gestire la complessita': l'incapsulamento riduce le dipendenze dai dettagli; l'ereditarieta' e il polimorfismo permettono di trattare in modo uniforme cose simili; la composizione costruisce oggetti complessi assemblando parti semplici, spesso in modo piu' flessibile dell'ereditarieta'.

16.3 Come funzionano internamente

In Python l'incapsulamento e' **per convenzione**: un underscore iniziale (`_nome`) segnala "privato per uso interno"; il doppio underscore attiva il *name mangling*. L'ereditarieta' si dichiara con `class Figlia(Madre)`; la risoluzione dei metodi segue l'MRO (Method Resolution Order). Il polimorfismo in Python e' **duck typing**: conta che l'oggetto abbia il metodo giusto, non il suo tipo esatto.

16.4 Sintassi Python

```
class Animale:
    def verso(self) -> str: return "..."
class Cane(Animale):
    def verso(self) -> str: return "Bau"      # override (polimorfismo)

for a in [Cane(), Animale()]:
    print(a.verso())      # "Bau", "..."
```

16.5 Esempi semplici

```
class Motore:
    def avvia(self): return "vroom"
```

```
class Auto:
    # COMPOSIZIONE: Auto HA-UN Motore
    def __init__(self):
        self.motore = Motore()
    def parti(self):
        return self.motore.avvia()
```

16.6 Applicazione nel progetto

Ereditarieta': `class LLMError(RuntimeError)` eredita da `RuntimeError` (a sua volta da `Exception`): e' l'esempio concreto nel codice. **Incapsulamento (per convenzione):** le funzioni "interne" hanno l'underscore — `_api_client`, `_ollama_chat`, `_api_chat`, `_extract_code_block`, `_restricted_env` — segnalando che non fanno parte dell'interfaccia pubblica del modulo. **Polimorfismo (duck typing):** `chat()` accetta qualsiasi lista di dizionari con `role/content`, indipendentemente da come e' stata costruita. **Composizione:** `llm_client` "ha" e usa un client (Ollama o OpenAI) a cui delega la comunicazione; i tool "hanno" e usano `llm_client` e `memory`. L'intero progetto e' costruito per **composizione di moduli** piu' che per ereditarieta'.

Approfondimento — "composition over inheritance"

Un principio di design molto citato: preferire la composizione all'ereditarieta' quando possibile, perche' piu' flessibile e meno fragile. Il progetto ne e' un esempio naturale: i tool *compongono* le funzionalita' di `llm_client` e `memory` invece di ereditare da una gerarchia.

16.7 Errori comuni

Attenzione

Abusare dell'ereditarieta' creando gerarchie profonde e rigide. Credere che l'underscore renda un attributo davvero inaccessibile: in Python e' solo una convenzione, resta accessibile.

16.8 Domande d'esame

Possibili domande

D: "Cos'e' il duck typing?" — **R:** "Se cammina come un'anatra e starnazza come un'anatra, e' un'anatra": conta che l'oggetto abbia i metodi richiesti, non il suo tipo.

D: "Differenza tra ereditarieta' e composizione?" — **R:** Ereditarieta' = "e'-un" (Cane e' un Animale); composizione = "ha-un" (Auto ha un Motore). La composizione e' spesso piu' flessibile.

D: "Dove c'e' ereditarieta' nel tuo progetto?" — **R:** In `LLMError(RuntimeError)`.

16.9 Riepilogo

Incapsulamento (nascondere), ereditarieta' ("e'-un"), polimorfismo (stesso metodo, comportamenti diversi), composizione ("ha-un"). Il progetto usa ereditarieta' per `LLMError`, incapsulamento per convenzione con l'underscore, e composizione tra moduli.

17. Eccezioni personalizzate

17.1 Definizione teorica

Un'**eccezione personalizzata** e' una classe che eredita da `Exception` (o una sua sottoclasse) per rappresentare un errore **specifico del dominio** dell'applicazione.

17.2 Perché esiste

Permette a chi usa il codice di catturare *quel* tipo di errore in modo mirato, separandolo dagli errori generici, e di trattare in modo uniforme guasti che internamente hanno cause diverse.

17.3 Come funziona internamente

Basta dichiarare una classe che estende `Exception`; può avere corpo vuoto (`pass`) o una docstring. Ereditando, ottiene tutto il comportamento delle eccezioni (messaggio, traceback, concatenamento con `from`).

17.4 Sintassi Python

```
class LLMError(RuntimeError):  
    """Errore di comunicazione con il modello."""
```

17.5 Esempi semplici

```
class SaldoInsufficiente(Exception):  
    pass  
raise SaldoInsufficiente("fondi non sufficienti")
```

17.6 Applicazione nel progetto

`LLMError` e' il cuore della gestione errori del modello. Le funzioni `_ollama_chat`, `_api_chat`, `_ollama_describe`, `_api_describe` catturano gli errori tecnici (rete assente, modello inesistente, chiave mancante) e li ri-sollevano **uniformemente** come `LLMError`. Così `main()` può catturare in un punto solo `except llm_client.LLMError` e mostrare un messaggio pulito, senza conoscere i dettagli di Ollama o dell'API. E' un ottimo esempio di uso *mirato* dell'ereditarietà.

17.7 Errori comuni

Attenzione

Ereditare da `BaseException` invece che da `Exception` (sbagliato: `BaseException` include anche `KeyboardInterrupt`). Creare troppe eccezioni inutili: se ne creano quando aggiungono valore semantico, come qui.

17.8 Domande d'esame

Possibili domande

D: "Perche' hai creato `LLMError`?" — **R:** Per esporre un errore di dominio uniforme, catturabile in un unico punto, nascondendo se il guasto viene da Ollama o dall'API.

D: "Da cosa eredita e perche'?" — **R:** Da `RuntimeError` (quindi da `Exception`), cosi' si comporta come una normale eccezione ma con un tipo riconoscibile.

17.9 Riepilogo

Le eccezioni personalizzate danno un nome agli errori di dominio. `LLMError` unifica i guasti dei due backend e semplifica la gestione centralizzata in `main()`.

18. Principi SOLID e design pattern

18.1 Definizione teorica

SOLID e' un acronimo di cinque principi di progettazione orientata agli oggetti. Un **design pattern** e' una soluzione ricorrente e collaudata a un problema di progettazione tipico.

Principio	Idea
S — Single Responsibility	Ogni unita' ha una sola ragione per cambiare (una responsabilita').
O — Open/Closed	Aperto all'estensione, chiuso alla modifica.
L — Liskov Substitution	Un sottotipo deve poter sostituire il tipo base senza sorprese.
I — Interface Segregation	Interfacce piccole e specifiche, non "onnicomprensive".
D — Dependency Inversion	Dipendere da astrazioni, non da implementazioni concrete.

18.2 Perche' esistono

Rendono il software piu' manutenibile, testabile ed estensibile, riducendo l'accoppiamento e aumentando la coesione. I pattern danno un **vocabolario comune**: dire "qui uso una Strategy" comunica molto in poche parole.

18.3 Come si applicano

SOLID nasce per l'OOP ma i suoi principi valgono anche per codice procedurale ben strutturato: una funzione con una sola responsabilita' rispetta l'idea della "S". I pattern piu' rilevanti qui sono **Strategy** (algoritmi intercambiabili dietro un'interfaccia comune), **Facade** (un'interfaccia semplice che nasconde un sottosistema complesso) e **Command** (incapsulare una richiesta come oggetto/handler).

18.4-18.6 Applicazione nel progetto

Single Responsibility: ogni modulo/funzione ha un compito unico — `auth` l'autenticazione, `memory` il DB, ogni tool una funzionalita'. E' il principio meglio rispettato dal progetto.

Strategy (di fatto): `chat()` sceglie tra due "strategie" di generazione (`_ollama_chat` vs `_api_chat`) dietro un'interfaccia unica. I tool non sanno quale strategia sia attiva: e' esattamente lo spirito del pattern Strategy.

Facade: `llm_client` e' una facciata che nasconde due sottosistemi complessi (Ollama e l'SDK OpenAI) dietro due funzioni semplici, `chat()` e `describe_image()`. I tool dialogano solo con la facciata.

Command / dispatch: in `main.py`, ogni comando e' associato a una funzione `cmd_*` tramite `set_defaults(func=...)`, e `args.func(args)` la invoca. E' una forma leggera del pattern Command, che sostituisce un lungo `if/elif`.

Dependency Inversion (in parte): i tool dipendono dall'astrazione "manda messaggi al modello" (`llm_client.chat`), non dai dettagli di Ollama o dell'API. Cambiare backend non tocca i tool.

18.7 Errori comuni

Attenzione

Applicare i pattern a forza dove non servono ("pattern-itis"). I pattern vanno riconosciuti quando emergono naturalmente, come qui, non imposti.

18.8 Domande d'esame

Possibili domande

D: "Il tuo progetto rispetta il Single Responsibility Principle?" — **R:** Sì: ogni modulo ha una responsabilità unica e ben delimitata.

D: "C'è qualche design pattern?" — **R:** Sostanzialmente sì: `llm_client` è una Facade con una scelta di backend in stile Strategy, e il dispatch dei comandi in `main.py` è una forma di Command.

D: "Cosa significa Open/Closed nel tuo caso?" — **R:** Aggiungere un terzo backend richiederebbe una nuova funzione `_xxx_chat` e un ramo in `chat()`, senza toccare i tool: estensione senza stravolgere il resto.

18.9 Riepilogo

SOLID = cinque principi per codice manutenibile; il progetto rispetta bene la "S" e in parte "O" e "D". Pattern presenti di fatto: Facade (`llm_client`), Strategy (scelta backend), Command (dispatch comandi).

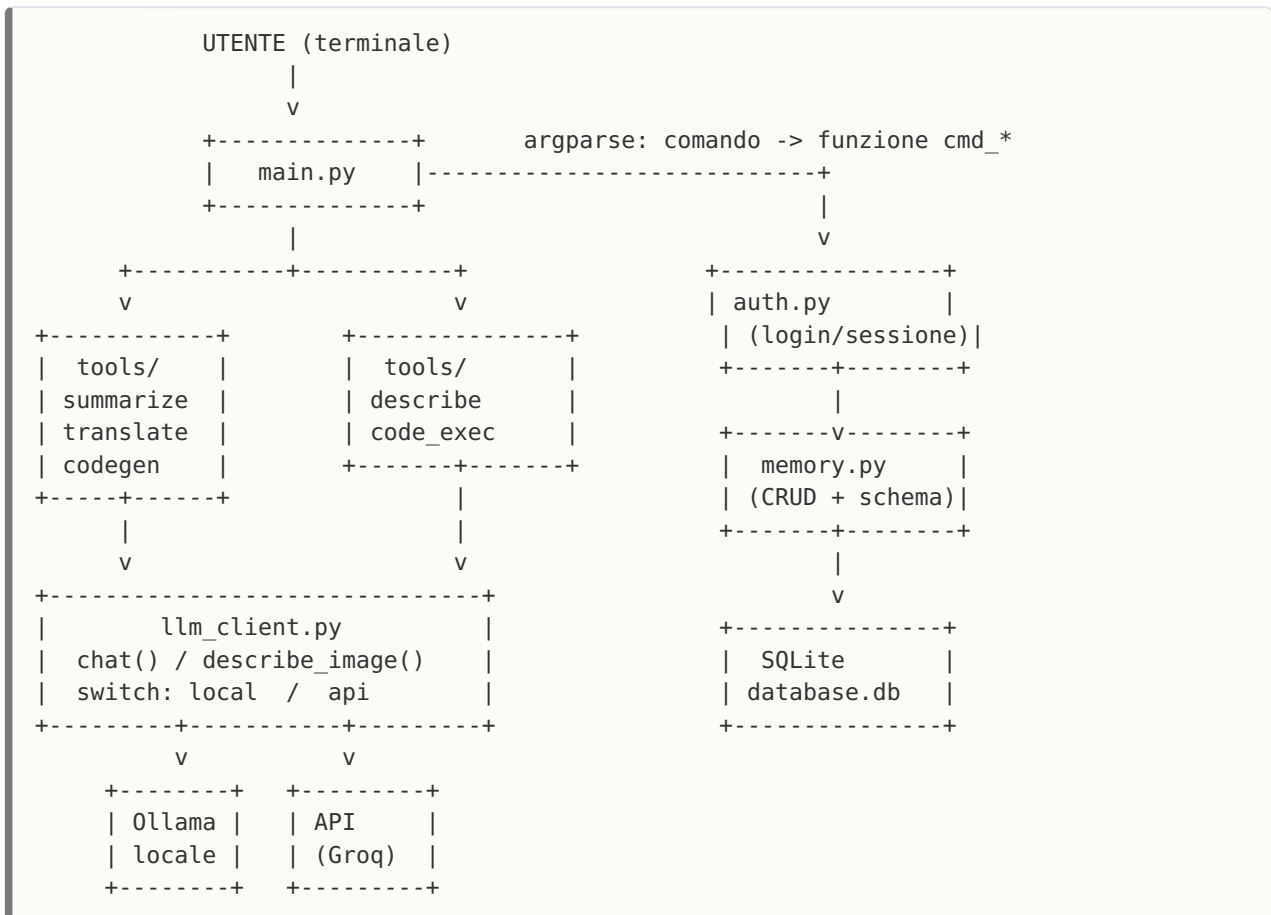
PARTE III

Analisi del progetto

19. Architettura e ciclo di vita

19.1 Visione d'insieme

Il progetto e' un'**applicazione CLI multi-utente**: si invoca da terminale con `python main.py <comando> [argomenti]`. L'architettura e' a **livelli** con responsabilita' separate: uno strato di *interfaccia* (CLI), uno di *strumenti* (i tool), uno di *accesso al modello* (llm_client) e uno di *persistenza* (auth + memory + SQLite).



19.2 Il ciclo di vita del programma

Ogni esecuzione e' **breve e indipendente** (un processo per comando):

1. **Avvio**: Python importa i moduli; `main.py` forza l'output in UTF-8 e le costanti di `llm_client` vengono lette dall'ambiente.
2. **Parsing**: `build_parser()` costruisce il parser; `parse_args()` riconosce comando e argomenti e sceglie la funzione `cmd_*`.
3. **Autorizzazione**: i comandi che operano sui dati chiamano `require_session()`, che legge la sessione da `db/.session.json`.
4. **Esecuzione**: la funzione del comando chiama il tool, che prepara l'input, invoca il modello via `llm_client` e registra l'operazione con `memory.add_message`.
5. **Output e uscita**: il risultato viene stampato; eventuali errori previsti sono intercettati in `main()` con messaggio pulito e `sys.exit(1)`. Il processo termina.

Da ricordare per l'esame

La frase chiave: **"ogni comando e' un processo separato che parte e finisce"**. Da qui derivano due scelte: la sessione salvata su file (per ricordare l'utente tra un comando e l'altro) e il database su disco (per la persistenza della cronologia).

19.3 Flusso dei dati (esempio: `summarize`)

```
file.pdf --> extract_text() --> testo
      (pypdf)           |
                      v
      [system prompt] + [user: "Riassumi:\n\n"+testo]
                      |
      llm_client.chat() --> (local|api) --> risposta
                      |
      memory.add_message(user, assistant)
                      v
                      stampa a schermo
```

20. Il file `main.py`

20.1 Perché esiste e ruolo

È il **punto d'ingresso** (entry point) e l'unico file pensato per essere eseguito direttamente. Ha la responsabilità dell'**interfaccia**: tradurre i comandi testuali dell'utente in chiamate alle funzioni giuste, e presentare i risultati.

20.2 Come comunica con gli altri file

Importa `auth`, `llm_client`, `memory` e i cinque tool. Non contiene logica applicativa: **orchestra**. È il livello più alto e dipende da tutti gli altri, ma nessuno dipende da lui (dipendenze a senso unico).

20.3 Funzioni principali

Funzione	Scopo	Ritorno
<code>require_session()</code>	Restituisce l'utente loggato o termina se assente.	<code>dict</code>
<code>cmd_login/logout/whoami</code>	Gestione sessione (non richiedono login preesistente).	None
<code>cmd_summarize/translate/describe/codegen/run/history</code>	Un handler per comando; chiama il tool e stampa.	None
<code>build_parser()</code>	Costruisce l'argparse con un subparser per comando.	<code>ArgumentParser</code>
<code>main()</code>	Esegue il comando e centralizza la gestione errori.	None

20.4 Il meccanismo di dispatch (punto forte)

Invece di un lungo `if command == "summarize": ... elif ...`, ogni subparser lega la propria funzione con `set_defaults(func=cmd_x)`, e `main()` fa semplicemente:

```
args = parser.parse_args()
args.func(args)    # chiama la funzione del comando scelto
```

Sfrutta le **funzioni come oggetti di prima classe** (cap. 8) ed è una forma del pattern Command (cap. 18). È elegante, estensibile e privo di duplicazione.

20.5 Dettaglio: la codifica UTF-8

All'inizio, `sys.stdout.reconfigure(encoding="utf-8")` forza l'output in UTF-8. Motivo: sui terminali Windows il default (cp1252/cp850) romperebbe gli accenti. Su Linux di solito è già UTF-8 e il blocco non fa nulla: è un accorgimento di **portabilità**.

20.6 Errori e robustezza

`main()` cattura `LLMError` e `(FileNotFoundError, ValueError)`: gli unici errori "previsti". Tutto il resto (bug veri) risale come traceback, così non si nascondono problemi reali. Buona pratica.

Domande tipiche

D: "Come fa il programma a sapere quale funzione eseguire?" — **R:** `Argparse` associa a ogni comando una funzione con `set_defaults(func=...)`; `main()` la invoca con `args.func(args)`.

D: "Cosa succede se lancio un comando senza essere loggato?" — **R:** `require_session()` non trova la sessione, stampa un avviso ed esce con codice 1.

21. Il file `llm_client.py`

21.1 Perché esiste e ruolo

E' l'**unico punto di contatto con il modello**. Incapsula la complessità di due backend (Ollama locale e API OpenAI-compatibile) dietro due funzioni: `chat()` per il testo e `describe_image()` per la vision. E' una **Facade** con una scelta di backend in stile **Strategy**.

21.2 Come comunica con gli altri file

E' usato da tutti i tool. Non conosce ne' il database ne' la CLI: riceve messaggi, restituisce testo. Questa indipendenza e' cio' che permette di cambiare backend senza toccare il resto.

21.3 Configurazione via ambiente

Le costanti di modulo leggono variabili d'ambiente con un default: `BACKEND`, `TEXT_MODEL`, `API_KEY`, `API_MODEL`, ecc. Così il comportamento e' configurabile **senza modificare il codice** (`os.environ.get("NOME", default)`).

21.4 Funzioni

Funzione	Scopo	Note
<code>chat(messages, ...)</code>	Invia una conversazione e restituisce il testo.	Pubblica; instrada al backend.
<code>describe_image(path, prompt)</code>	Descrive un'immagine.	Pubblica; instrada al backend.
<code>_api_client()</code>	Crea il client OpenAI (import pigro).	Interna; solleva <code>LLMError</code> se manca la chiave.
<code>_ollama_chat</code> / <code>_api_chat</code>	Implementazioni concrete della chat.	Interne (underscore).
<code>_ollama_describe</code> / <code>_api_describe</code>	Implementazioni della vision.	La versione API usa base64.

21.5 Dettaglio: la scelta del backend

```
def chat(messages, model=None, temperature=0.2, **options):
    if BACKEND == "api":
        return _api_chat(messages, model or API_MODEL, temperature)
    return _ollama_chat(messages, model or TEXT_MODEL, temperature, **options)
```

I messaggi `{role, content}` hanno lo **stesso formato** per Ollama e per le API OpenAI-compatibili: nessuna conversione. La vision e' l'unico punto che diverge: l'API vuole l'immagine come **data URI base64**, Ollama usa il parametro nativo `images`.

21.6 Import pigri e gestione errori

`import ollama` e `from openai import OpenAI` sono **dentro** le funzioni: chi usa un solo backend non deve installare la libreria dell'altro. Ogni chiamata e' protetta da `try/except` che traduce i guasti in `LLMError` con `from exc`.

Domande tipiche

D: "Come faresti per aggiungere un terzo provider?" — **R:** Aggiungere una funzione `_xxx_chat` e un ramo in `chat()`; i tool non cambierebbero (principio Open/Closed).

D: "Perche' gli import sono dentro le funzioni?" — **R:** Import pigro: carica la libreria solo del backend usato, riducendo le dipendenze obbligatorie.

D: "Cos'e' una Facade e dove la vedi qui?" — **R:** Un'interfaccia semplice davanti a un sottosistema complesso: `llm_client` nasconde Ollama e l'SDK OpenAI dietro `chat()/describe_image()`.

22. Il package `tools/` (i cinque strumenti)

22.1 Perché esiste e ruolo

`tools/` è un **package** (cartella con `__init__.py`) che raccoglie i cinque strumenti, uno per funzionalità. Ogni tool ha la stessa struttura logica: **prepara l'input** → **chiama il modello** → **interpreta la risposta** → **salva/registra** → **restituisce**. Questa uniformità è un esempio di buona coesione.

22.2 `summarize.py`

Due funzioni. `extract_text(path) -> str` legge `.txt` (con `read_text`) o `.pdf` (con `pypdf`), sollevando `FileNotFoundError/ValueError` sugli input errati; è riuscita anche da `translate`. `summarize_file(path, user_id, max_length=None) -> str` costruisce due messaggi (system + user), chiama `llm_client.chat`, tronca eventualmente a `max_length` e registra l'operazione. È un tool **stateless**: non inietta la cronologia nel prompt.

22.3 `translate.py`

`translate_file(path, target_lang, user_id) -> tuple[str, Path]`: estrae il testo (riusando `extract_text` — buon riuso), costruisce il prompt, chiama il modello, salva il risultato accanto all'originale (`documento.english.txt`) e registra. Restituisce sia il testo sia il percorso: una tupla come **record a due campi**.

22.4 `describe_image.py`

`describe(image_path, user_id) -> str`: valida l'esistenza e il formato (set `SUPPORTED_EXTENSIONS`), poi delega a `llm_client.describe_image`. È l'unico tool che usa il modello *vision* (multimodale).

22.5 `codegen.py`

`generate(prompt, user_id, lang="python", cot=False) -> str`: costruisce il prompt (con o senza Chain-of-Thought), chiama il modello ed estrae il codice dal blocco markdown con `_extract_code_block` (una regex, con fallback all'intera risposta). `save_to_file(code, output, lang) -> Path`: salva su file, usando un nome con **timestamp** se `output` non è fornito, e sceglie l'estensione dal dizionario `LANG_EXTENSIONS`.

22.6 `code_exec.py`

È il tool più delicato: esegue codice. `run_file(path, timeout, input_text) -> tuple[str, str, int]` avvia un **sottoprocesso isolato** con `subprocess.run([sys.executable, "-I", file], ...)`, timeout ed **ambiente ristretto** (variabili diverse per Windows e Linux). Restituisce `(stdout, stderr, returncode)`; su timeout, `returncode -1`. `run_and_record` aggiunge la registrazione in cronologia.

Approfondimento — isolamento dell'esecuzione

Il flag `-I` (isolated mode) fa ignorare a Python le variabili d'ambiente `PYTHON*` e i pacchetti utente; il timeout evita loop infiniti; l'ambiente ridotto limita cio' che il codice puo' vedere. E' un isolamento **a livello di processo**, non un sandbox completo (quello richiederebbe container/VM): sufficiente in ambito didattico, da non usare con codice non fidato in produzione. Saperlo dire con precisione e' un ottimo segnale all'esame.

22.7 Errori comuni evidenziati

Attenzione

Eseguire codice generato da un modello e' intrinsecamente rischioso: il progetto lo circonda ma lo documenta come limite noto. E' corretto **dichiarare i limiti** invece di fingere una sicurezza che non c'e'.

Domande tipiche

D: "Come isoli l'esecuzione del codice?" — **R:** Sottoprocesso separato con `-I`, timeout e ambiente ridotto; e' isolamento di processo, non un sandbox completo.

D: "Perche' `extract_text` sta in `summarize` ma la usa anche `translate`?" — **R:** Per riuso (DRY): la logica di lettura file e' una sola, importata dove serve.

23. `memory.py` e il database

23.1 Perché esiste e ruolo

E' lo **strato di persistenza**: gestisce la connessione al database SQLite e le operazioni su utenti e cronologia. Usa `sqlite3` della libreria standard: nessun server esterno, il database e' un unico file `db/database.db`.

23.2 Lo schema (due tabelle)

```
users                                memory
+---+-----+-----+             +---+-----+-----+-----+-----+
+-----+
| id | username | password | | id | user_id | role | content | tool_used | timestamp |
|
+---+-----+-----+             +---+-----+-----+-----+-----+
+-----+
      (unica)                        (FK -> users.id)
```

La tabella `memory` ha una **foreign key** verso `users`: ogni messaggio appartiene a un utente. Questo lega la cronologia all'identita' (relazione uno-a-molti).

23.3 Funzioni

Funzione	Scopo	SQL
<code>get_connection() -> Connection</code>	Apri la connessione, applica lo schema, esegue la migrazione.	<code>executescript</code> , <code>PRAGMA</code> , <code>ALTER</code>
<code>add_message(...) -> None</code>	Salva un messaggio nella cronologia.	<code>INSERT</code>
<code>get_recent(user_id, n) -> list[dict]</code>	Legge gli ultimi N messaggi.	<code>SELECT ... ORDER BY id DESC LIMIT</code>

23.4 Auto-inizializzazione e migrazione

`get_connection` esegue lo schema a ogni apertura: grazie a `CREATE TABLE IF NOT EXISTS` l'operazione e' **idempotente** e il DB si crea da solo al primo uso. Contiene inoltre una **migrazione**: se manca la colonna `password` (DB vecchi), la aggiunge con `ALTER TABLE` e assegna agli utenti esistenti una password iniziale pari allo username. E' un modo pulito per far evolvere lo schema senza rompere i dati esistenti.

23.5 Dettagli tecnici che dimostrano padronanza

- **Query parametrizzate (?)**: `conn.execute("... VALUES (?, ?)", (a, b))`.
Prevencono la **SQL injection** e gestiscono l'escaping. Mai concatenare stringhe nelle query.

- **row_factory = sqlite3.Row**: permette l'accesso alle colonne per nome (`row["username"]`) invece che per indice.
- **ORDER BY id DESC + reversed**: prende gli N piu' recenti ma li restituisce in ordine cronologico naturale.
- **try/finally con close()**: la connessione viene sempre chiusa.

Domande tipiche

D: "Perche' usi i ? nelle query?" — **R:** Sono parametri: prevengono la SQL injection e l'errore di escaping, separando i dati dal comando SQL.

D: "Come si inizializza il database?" — **R:** Da solo: lo schema con `CREATE TABLE IF NOT EXISTS` viene applicato a ogni connessione ed e' idempotente.

D: "Cos'e' una foreign key e dove la usi?" — **R:** Un vincolo che collega una tabella all'altra: `memory.user_id` riferisce `users.id` , legando ogni messaggio a un utente.

24. `auth.py` e la sessione

24.1 Perché esiste e ruolo

Gestisce **identità** e **sessione**: login/registrazione con password e memorizzazione dell'utente attivo. Le credenziali stanno nel DB (tabella `users`), l'utente attivo in un file JSON.

24.2 Le tre funzioni

Funzione	Scopo	Ritorno
<code>login(username, password) -> int</code>	Registra (primo accesso) o verifica; salva la sessione.	<code>user_id</code>
<code>logout() -> None</code>	Elimina il file di sessione.	<code>None</code>
<code>current_user() -> dict None</code>	Legge l'utente attivo dalla sessione.	<code>dict</code> o <code>None</code>

24.3 Perché la sessione è un file

Poiché ogni comando è un processo separato, senza una memoria esterna il programma non ricorderebbe chi è loggato. La sessione è un piccolo JSON (`db/.session.json`) con `user_id` e `username`: semplice, leggibile e sufficiente. `logout` lo cancella; `current_user` lo legge, restituendo `None` se assente o corrotto.

24.4 Il login

```
row = conn.execute("SELECT id, password FROM users WHERE username = ?",
(username,)).fetchone()
if row is None:
    # primo accesso: registra
    ... INSERT ...
elif password == row["password"]:
    # accesso: verifica
    user_id = row["id"]
else:
    raise ValueError("Password errata.")
```

Attenzione — cattiva pratica dichiarata

La password è salvata **in chiaro**. È una semplificazione didattica esplicitamente documentata. In produzione si userebbe un **hash con salt** (es. `hashlib.pbkdf2_hmac`) e si confronterebbe l'hash, mai la password in chiaro. Riconoscere e motivare questo limite all'esame vale più che nascondere.

Domande tipiche

D: "Come miglioreresti la sicurezza delle password?" — **R:** Salvando un hash con salt invece del testo in chiaro, e confrontando gli hash.

D: "Perché la sessione non è nel database?" — **R:** È un dato minimo e volatile ("chi è

attivo ora"); un file JSON e' piu' semplice e adatto, il DB resta per dati persistenti come utenti e cronologia.

PARTE IV

Ingegneria del software

25. Organizzazione, dipendenze, virtual environment

25.1 Struttura del progetto

```
agente/  
  main.py           entry point (CLI)  
  llm_client.py     accesso al modello (facade)  
  memory.py         database (persistenza)  
  auth.py           login e sessione  
  tools/            package degli strumenti  
    __init__.py  
    summarize.py    translate.py  describe_image.py  
    codegen.py      code_exec.py  
  db/  
    schema.sql      definizione tabelle  
    database.db     dati (generato)  
  requirements.txt  dipendenze  
  README.md         istruzioni
```

La struttura riflette le responsabilità: un file per compito, i tool raggruppati in un package. E' immediatamente leggibile: già dai nomi si capisce l'architettura.

25.2 Gestione delle dipendenze

Le dipendenze esterne stanno in `requirements.txt` (`ollama`, `pypdf`, `openai`), installabili con `pip install -r requirements.txt`. Elencare le dipendenze in un file **riproducibile** e' fondamentale: chiunque puo' ricreare l'ambiente. Il resto (`sqlite3`, `argparse`, `json`, `subprocess`, `pathlib`, `base64`, `re`) e' **libreria standard**, quindi non va installato.

25.3 Virtual environment

Un **virtual environment** e' una cartella con un interprete Python isolato e le sue librerie, separate da quelle di sistema. Serve a evitare conflitti di versione tra progetti diversi.

```
python3 -m venv venv          # crea l'ambiente  
source venv/bin/activate      # attiva (Linux/macOS)  
# venv\Scripts\activate       # attiva (Windows)  
pip install -r requirements.txt
```

Da ricordare per l'esame

Il "perche'" del venv: **isolamento e riproducibilita'**. Ogni progetto ha le sue dipendenze, con le versioni giuste, senza sporcare il Python di sistema o gli altri progetti.

Domande tipiche

D: "A cosa serve un virtual environment?" — **R:** A isolare le dipendenze di un progetto, evitando conflitti di versione e rendendo l'ambiente riproducibile.

D: "Cosa contiene `requirements.txt` ?" — **R:** L'elenco delle librerie esterne necessarie, per reinstallarle in modo riproducibile.

26. Buone e cattive pratiche nel progetto

Un professore apprezza chi sa **giudicare criticamente** il proprio codice. Ecco un bilancio onesto.

26.1 Buone pratiche

Pratica	Dove	Perche' e' buona
Separazione delle responsabilita'	Tutta la struttura	Ogni modulo un compito: manutenibile e leggibile.
Type hints completi	Ogni funzione	Documentano il contratto; utili agli strumenti statici.
Docstring (Args/Returns/Raises)	Ogni funzione	Spiegano scopo e tipi di input/output.
Query parametrizzate	memory.py	Prevengono la SQL injection.
Gestione errori centralizzata	main() + LLMError	Messaggi puliti, niente traceback all'utente.
Configurazione via ambiente	llm_client.py	Cambiare backend/modello senza toccare il codice.
Portabilita' Windows/Linux	UTF-8, env per OS	Gira su entrambi i sistemi.

26.2 Cattive pratiche (dichiarate) e come si risolverebbero

Limite	Rischio	Soluzione corretta
Password in chiaro	Sicurezza	Hash con salt (<code>pbkdf2_hmac</code>).
Chiave API nel codice	Segreto esposto	Variabile d'ambiente o file <code>.env</code> non versionato.
Esecuzione codice non in sandbox reale	Codice non fidato	Container/VM con permessi minimi.
Assenza di test automatici	Regressioni	Suite <code>pytest</code> (cap. 27).
Stile procedurale	Punti OOP	Incapsulare in classi <code>LLMClient</code> / <code>Database</code> / <code>Tool</code> .

Da ricordare per l'esame

Presentare i limiti come **scelte consapevoli con alternative note** ("ho fatto cosi' per semplicita' didattica; in produzione userei X") e' esattamente l'atteggiamento che un docente vuole vedere.

27. Testing e logging

27.1 Testing (assente nel progetto: teoria + come aggiungerlo)

Il **testing automatico** verifica che il codice si comporti come atteso, tramite test ripetibili. I **test unitari** isolano una singola funzione; usano spesso **mock** per sostituire dipendenze esterne (rete, modello). Il framework piu' comune e' `pytest`.

```
def test_extract_summary(tmp_path):
    f = tmp_path / "x.txt"; f.write_text("ciao")
    assert extract_text(str(f)) == "ciao"
```

Nel progetto sarebbero facili da testare le funzioni pure e deterministiche: `extract_text`, `_extract_code_block`, `save_to_file`, `run_file` (con codice noto), la logica di `login` su un DB temporaneo. Le chiamate al modello si testerebbero con un **mock** di `llm_client.chat`, per non dipendere dalla rete.

27.2 Logging (assente: teoria + come aggiungerlo)

Il **logging** registra eventi del programma con livelli di gravita' (DEBUG, INFO, WARNING, ERROR, CRITICAL) tramite il modulo standard `logging`. E' preferibile a `print()` perche' e' configurabile (destinazione, livello, formato) e separa i messaggi diagnostici dall'output per l'utente.

```
import logging
log = logging.getLogger(__name__)
log.info("Backend attivo: %s", BACKEND)
```

Nel progetto, `print()` e' usato per l'output all'utente (corretto, e' un'interfaccia CLI), ma per la diagnostica interna (quale backend, quale modello, errori dettagliati) il logging sarebbe l'evoluzione naturale.

Domande tipiche

D: "Come testeresti una funzione che chiama il modello?" — **R:** Sostituendo `llm_client.chat` con un mock che restituisce una risposta finta, cosi' il test non dipende dalla rete.

D: "Differenza tra `print` e `logging`?" — **R:** `print` e' output semplice; `logging` ha livelli, formattazione e destinazioni configurabili, adatto alla diagnostica.

28. Domande d'esame trasversali

Domande "a piacere" che uniscono piu' argomenti. Usa le risposte come traccia.

Domande e risposte modello

1. "Mi spieghi il tuo progetto in un minuto."

— E' un assistente AI da terminale, multi-utente. Da CLI puoi riassumere file, tradurre, descrivere immagini e generare/eseguire codice. Il modello gira in locale (Ollama) o via API (Groq), scelto con una variabile d'ambiente. Utenti e cronologia sono su SQLite. L'architettura e' a livelli: CLI → tool → client del modello → persistenza.

2. "Qual e' la parte piu' interessante dal punto di vista progettuale?"

— Il modulo `llm_client`: e' una Facade che nasconde due backend dietro `chat()` / `describe_image()`, scegliendoli in stile Strategy. I tool non sanno quale backend sia attivo, quindi cambiarlo non li tocca (basso accoppiamento).

3. "Come garantisci che il programma non crashi con input sbagliati?"

— Le funzioni sollevano eccezioni specifiche (`FileNotFoundError`, `ValueError`, `LLMError`), e `main()` le cattura in un punto solo stampando un messaggio pulito ed uscendo con codice 1.

4. "Perche' SQLite e non un file di testo?"

— Perche' servono relazioni (utenti↔cronologia), query strutturate (ultimi N per utente) e integrita' (foreign key, valori parametrizzati). SQLite offre tutto questo in un unico file, senza server.

5. "Dove sono i concetti OOP?"

— Ereditarieta' in `LLMError(RuntimeError)`; incapsulamento per convenzione con le funzioni `_`; polimorfismo come duck typing sui messaggi; composizione tra moduli. La logica e' procedurale, ma incapsulabile in classi `LLMClient` / `Database` / `Tool`.

6. "Come lo rendi eseguibile su un altro computer?"

— Virtual environment + `pip install -r requirements.txt`; per il modello, o Ollama in locale o una chiave API. Nessun altro setup, i percorsi usano `pathlib` e sono portabili.

7. "Un difetto del progetto e come lo correggeresti?"

— Le password in chiaro: userei un hash con salt (`pbkdf2_hmac`). E la mancanza di test: aggiungerei `pytest` sulle funzioni pure e un mock del modello.

Glossario

Termine	Significato
Oggetto	Dato in memoria con identita', tipo e valore.
Riferimento	Legame tra un nome e un oggetto.
Mutabile / immutabile	Modificabile / non modificabile dopo la creazione.
Hashable	Oggetto con hash stabile, usabile come chiave/elemento di set.
Truthiness	Valore di verita' di un oggetto in contesto booleano.
Comprensione	Sintassi concisa per costruire una collezione.
Generatore	Iteratore pigro, produce valori su richiesta.
Scope (LEGB)	Regole di visibilita' dei nomi: Local, Enclosing, Global, Built-in.
Type hint	Annotazione di tipo, non vincolante a runtime.
Eccezione	Oggetto che rappresenta un errore; gestita con try/except.
Context manager	Oggetto usato con <code>with</code> , garantisce il rilascio risorse.
Modulo / package	File <code>.py</code> / cartella di moduli.
Facade	Interfaccia semplice davanti a un sottosistema complesso.
Strategy	Algoritmi intercambiabili dietro un'interfaccia comune.
Duck typing	Conta il comportamento (i metodi) dell'oggetto, non il tipo.
Idempotente	Operazione che ripetuta da' lo stesso risultato (es. lo schema del DB).
Foreign key	Vincolo che collega una tabella a un'altra.
Query parametrizzata	Query con segnaposto <code>?</code> , previene la SQL injection.
Virtual environment	Interprete Python isolato con le proprie dipendenze.

Fine del manuale — Buono studio e in bocca al lupo per l'esame.